



MVP - Engenharia de Dados

Produtividade de equipes na indústria de confecção

Izis Dornelas de Paula

Email: izisdornelas@gmail.com

GitHub: <https://github.com/izisdornelas>

Discord: izisdornelas

Sprint: Engenharia de Dados (40530010057_20260_01)

Repositório: <https://github.com/izisdornelas/mvp-engenharia-dados-produtividade>

Este projeto reúne dados de produtividade de equipes, preserva a fonte original, aplica regras de padronização e disponibiliza tabelas para responder a perguntas de gestão. O pipeline foi executado no Databricks Free Edition sobre 1.197 registros de uma base pública da UCI Machine Learning Repository.

Sumário

1 Contexto de Negócios e Perguntas	2
Fonte dos dados	2
2 Carga dos Dados	2
3 Modelagem e Catálogo de Dados	4
Catálogo dos campos de origem e da tabela Silver	4
Campos adicionais da Bronze e da Silver	5
Catálogo das três tabelas Gold	5
4 Pipeline de Dados	6
5 Qualidade de Dados	7
6 Análise de Dados	10
Cumprimento das metas por setor	10
Variação ao longo do período e entre equipes	11
Indicadores operacionais segundo o cumprimento da meta	12
Síntese das perguntas e respostas	13
Considerações finais	13
Pontos de melhoria	13
7 Autoavaliação	14
Referências	14

1 Contexto de Negócios e Perguntas

Em uma indústria de confecção, diferentes equipes recebem metas de produtividade e registram o resultado de seu trabalho. Para acompanhar essas operações, é útil identificar onde a meta é cumprida e como variam os resultados por setor, equipe e data. O objetivo deste MVP é transformar um arquivo público com registros de produtividade em tabelas consultáveis no Databricks e utilizá-las para examinar o cumprimento das metas.

As perguntas que orientaram a modelagem e as consultas são:

1. Qual proporção de registros atinge ou supera a meta de produtividade em cada setor?
2. Como a produtividade realizada e o cumprimento da meta variam entre equipes e ao longo das datas disponíveis?
3. Como se distribuem horas extras, tempo de inatividade e trabalho em andamento nos registros que atingem e não atingem a meta?

Fonte dos dados

Foi utilizado o arquivo `garments_worker_productivity.csv` do conjunto Productivity Prediction of Garment Employees, disponibilizado pela UCI Machine Learning Repository (DOI 10.24432/C51S6D). A página da UCI informa licença Creative Commons Attribution 4.0 International (CC BY 4.0). Os registros se referem à indústria de confecção.

The screenshot shows the UCI Machine Learning Repository page for the 'Productivity Prediction of Garment Employees' dataset. The page includes a header with the UCI logo and navigation links. The main content area features a blue header with the dataset name and a 'Donated on 8/2/2020' note. Below this, there is a description of the dataset and a table with characteristics. To the right, there are buttons for 'DOWNLOAD', 'IMPORT IN PYTHON', and 'CITE', along with citation and view counts. At the bottom, there is a 'License' section with the Creative Commons Attribution 4.0 International license details.

Dataset Characteristics	Subject Area	Associated Tasks
Multivariate, Time-Series	Business	Classification, Regression

Feature Type	# Instances	# Features
Integer, Real	1197	14

Dataset Information

Additional Information

The Garment Industry is one of the key examples of the industrial globalization of this modern era. It is a highly labour-intensive industry with lots of manual processes. Satisfying the huge global demand for garment products is mostly dependent on the production and delivery performance of the employees in the garment manufacturing companies. So, it is highly desirable among the decision makers in the garments industry to track, analyse and predict the productivity performance of the working teams in their factories. This dataset can be used for regression purpose by predicting the productivity range (0-1) or for classification purpose by transforming the productivity range (0-1) into different classes.

License

This dataset is licensed under a [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#) license.

This allows for the sharing and adaptation of the datasets for any purpose, provided that the appropriate credit is given.

Figura 1 — Página do conjunto Productivity Prediction of Garment Employees na UCI Machine Learning Repository. Fonte: UCI (2020).

O CSV contém 1.197 linhas e 15 colunas: `date`, `quarter`, `department`, `day`, `team`, `targeted_productivity`, `smv`, `wip`, `over_time`, `incentive`, `idle_time`, `idle_men`, `no_of_style_change`, `no_of_workers` e `actual_productivity`. Há 59 datas de 1º de janeiro a 11 de março de 2015. Cada linha descreve o registro de uma equipe em um setor e uma data. A chave composta por data, setor padronizado e equipe não apresenta repetições no arquivo analisado.

Fonte e licença: <https://archive.ics.uci.edu/dataset/597/productivity+prediction+of+garment+employees>

2 Carga dos Dados

O CSV foi obtido na página da UCI e enviado manualmente ao volume `entrada_uci` do esquema `workspace.mvp_produtividade`, no Databricks Free Edition. O caminho utilizado pelo notebook é

/Volumes/workspace/mvp_produtividade/entrada_uci/garments_worker_productivity.csv. O arquivo permanece nesse volume como origem para eventuais reexecuções.

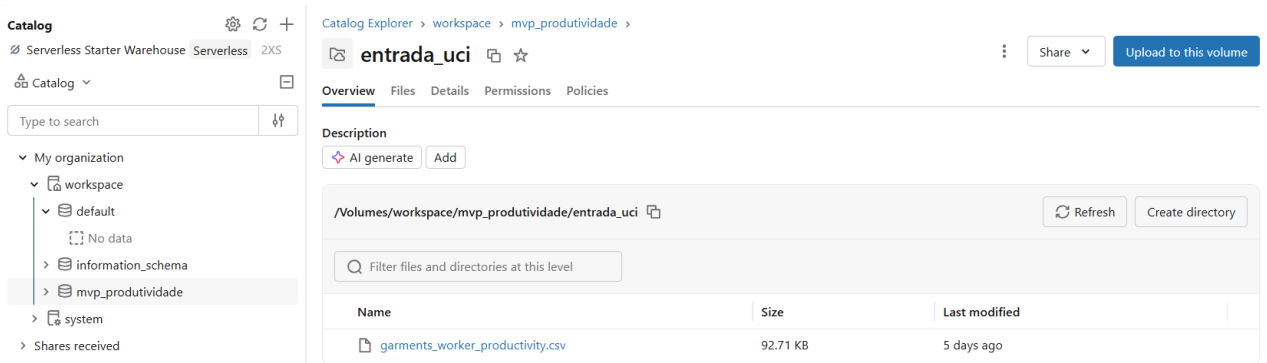


Figura 2 — Arquivo garments_worker_productivity.csv armazenado no volume entrada_uci do Databricks.

Fonte: elaboração própria.

```
CATALOG = "workspace"
SCHEMA = "mvp_produtividade"
VOLUME = "entrada_uci"
CSV_NAME = "garments_worker_productivity.csv"

spark.sql(f"CREATE SCHEMA IF NOT EXISTS `{CATALOG}`.`{SCHEMA}`")
spark.sql(f"CREATE VOLUME IF NOT EXISTS `{CATALOG}`.`{SCHEMA}`.`{VOLUME}`")
CSV_PATH = f"/Volumes/{CATALOG}/{SCHEMA}/{VOLUME}/{CSV_NAME}"
TABLE_PREFIX = f"{CATALOG}.{SCHEMA}"
```

Figura 3 — Configuração do esquema, do volume e do caminho do CSV no Databricks.

Fonte: elaboração própria.

No notebook 01_pipeline_produtividade.py, o Spark lê o cabeçalho sem inferir tipos e confere os 15 nomes de coluna esperados. Os campos são preservados como texto na tabela Delta bronze_produtividade; apenas source_file, que identifica o CSV, e ingested_at, que registra a ingestão, são acrescentados. A execução registrou 1.197 linhas nessa tabela. O modo FAILFAST interrompe a leitura de registros CSV malformados; uma validação separada confere os nomes e a ordem das 15 colunas.

```
raw = spark.read.option("header", True).option("inferSchema", False).option("mode", "FAILFAST").csv(CSV_PATH)
source_columns = ["date", "quarter", "department", "day", "team", "targeted_productivity", "smv", "wip", "over_time", "incentive", "idle_time", "idle_men",
"no_of_style_change", "no_of_workers", "actual_productivity"]
assert raw.columns == source_columns, f"Colunas inesperadas no CSV: {raw.columns}"
bronze = raw.withColumn("source_file", F.lit(CSV_NAME)).withColumn("ingested_at", F.current_timestamp())
bronze.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(f"{TABLE_PREFIX}.bronze_produtividade")
print("Linhas Bronze:", spark.table(f"{TABLE_PREFIX}.bronze_produtividade").count())
display(spark.table(f"{TABLE_PREFIX}.bronze_produtividade").limit(5))
```

Linhas Bronze: 1197

	date	quarter	department	day	team	targeted_productivity	smv	wip	over_time	incentive
1	1/1/2015	Quarter1	sweing	Thursday	8	0.8	26.16	1108	7080	98
2	1/1/2015	Quarter1	finishing	Thursday	1	0.75	3.94	null	960	0
3	1/1/2015	Quarter1	sweing	Thursday	11	0.8	11.41	968	3660	50
4	1/1/2015	Quarter1	sweing	Thursday	12	0.8	11.41	968	3660	50
5	1/1/2015	Quarter1	sweing	Thursday	6	0.8	25.9	1170	1920	50

Figura 4 — Leitura do CSV, gravação da tabela Bronze e conferência dos 1.197 registros. Fonte: elaboração própria.

Código do pipeline: https://github.com/izisdornelas/mvp-engenharia-dados-produtividade/blob/main/01_pipeline_produtividade.py

3 Modelagem e Catálogo de Dados

O modelo organiza os dados em uma tabela de entrada, uma tabela detalhada tratada e três tabelas de agregação. Cada agregação responde a uma necessidade de consulta; não há tabelas de dimensões nem junções, pois o arquivo de origem contém uma única relação de registros de produtividade.

Tabela no esquema workspace.mvp_produtividade	Grão e conteúdo	Linhas
bronze_produtividade	Uma linha do CSV; valores originais e metadados de ingestão.	1.197
silver_produtividade	Data, setor e equipe; tipos corrigidos e indicadores de validação.	1.197
gold_produtividade_setor	Um setor; síntese das metas e indicadores operacionais.	2
gold_produtividade_dia_setor	Uma data e um setor; evolução dos indicadores.	118
gold_produtividade_equipe	Um setor e uma equipe; comparação entre equipes.	24

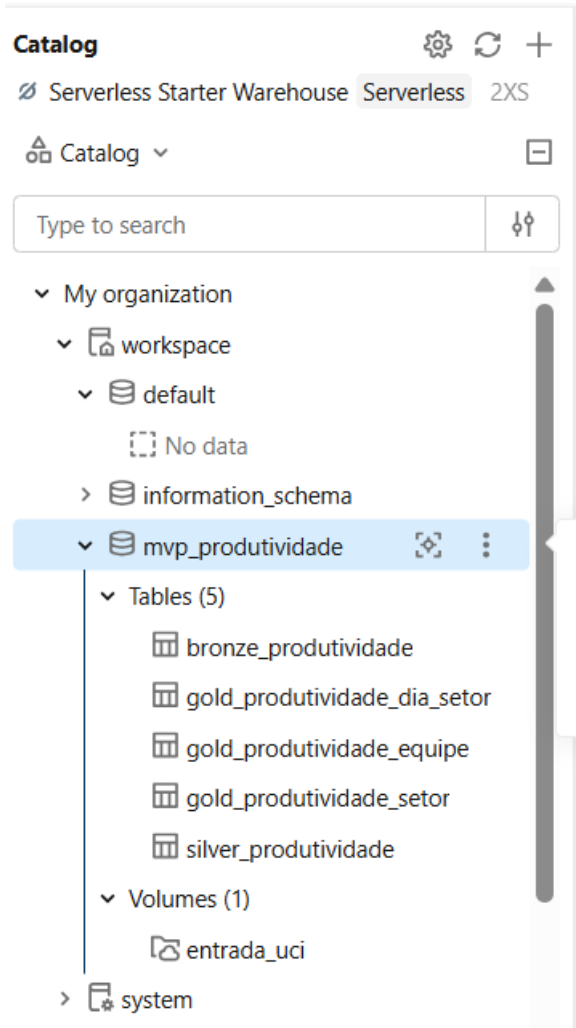


Figura 5 — Cinco tabelas persistidas no esquema workspace.mvp_produtividade do Databricks. Fonte: elaboração própria.

Catálogo dos campos de origem e da tabela Silver

Na Bronze, os 15 campos originais abaixo têm tipo STRING. Na Silver, os nomes e tipos resultam das conversões indicadas. Os intervalos são os valores observados neste arquivo e não regras universais para futuros dados.

Origem e nome na Silver	Tipo Silver	Definição e domínio observado
date → data_registro	DATE	Data da produção; de 01/01 a 11/03/2015.
quarter → quarter	STRING	Parte do mês. Quarter1, Quarter2, Quarter3, Quarter4 e Quarter5 no arquivo.

Origem e nome na Silver	Tipo Silver	Definição e domínio observado
department → setor_original, setor	STRING	Texto recebido e setor padronizado: sewing ou finishing.
day → day	STRING	Dia da semana, de Monday a Sunday, exceto Friday no período observado.
team → equipe	INT	Identificação numérica da equipe, de 1 a 12.
targeted_productivity	DOUBLE	Produtividade prevista como meta, de 0,07 a 0,8.
smv	DOUBLE	Standard Minute Value, de 2,9 a 54,56.
wip	DOUBLE	Trabalho em andamento; 7 a 23.122 quando informado.
over_time	DOUBLE	Tempo extra da equipe em minutos, de 0 a 25.920.
incentive	DOUBLE	Incentivo em BDT, de 0 a 3.600.
idle_time	DOUBLE	Tempo de interrupção, de 0 a 300; unidade não especificada pela UCI.
idle_men	DOUBLE	Número de pessoas ociosas, de 0 a 45.
no_of_style_change	DOUBLE	Mudanças de estilo do produto, de 0 a 2.
no_of_workers	DOUBLE	Número de trabalhadores, de 2 a 89; há valores fracionários na fonte.
actual_productivity	DOUBLE	Produtividade realizada, de 0,233705476 a 1,1204375.

A Bronze ainda contém source_file (STRING) e ingested_at (TIMESTAMP). A Silver preserva source_file, quarter, day e as medidas, além de conter atingiu_meta, produtividade_acima_de_um e dia_confere (BOOLEAN). O primeiro indicador compara produtividade realizada com a meta; o segundo sinaliza valores acima de 1; o terceiro confere o dia da semana com a data convertida.

Campos adicionais da Bronze e da Silver

Campo	Tabela e tipo	Origem, significado e domínio
source_file	Bronze e Silver · STRING	Nome do CSV carregado; nesta carga, garments_worker_productivity.csv.
ingested_at	Bronze · TIMESTAMP	Data e hora atribuídas pelo pipeline à gravação da Bronze.
setor_original	Silver · STRING	Cópia de department, preservando a grafia recebida.
setor	Silver · STRING	department sem espaços nas extremidades, em minúsculas, com sweing corrigido para sewing; valores finais: sewing e finishing.
atingiu_meta	Silver · BOOLEAN	Verdadeiro quando actual_productivity >= targeted_productivity.
produtividade_acima_de_um	Silver · BOOLEAN	Verdadeiro quando actual_productivity > 1; sinaliza o valor sem alterá-lo.
dia_confere	Silver · BOOLEAN	Confere se o dia da semana calculado de data_registro corresponde a day.

Catálogo das três tabelas Gold

As tabelas Gold são calculadas exclusivamente a partir da Silver, sem junções. A linhagem dos dados começa no CSV armazenado no volume entrada_uci, passa pela Bronze, que preserva os dados recebidos, e pela Silver, que converte tipos, padroniza o setor e cria os indicadores. A gold_produtividade_setor agrupa por setor (STRING); a gold_produtividade_dia_setor, por data_registro (DATE) e setor (STRING); e a gold_produtividade_equipe, por setor (STRING) e equipe (INT). Os nove campos restantes são calculados para cada grupo e estão descritos na tabela a seguir:

Campo	Tipo	Definição
registros	BIGINT	Quantidade de registros do grupo.
metas_atingidas	BIGINT	Quantidade de registros com atingiu_meta verdadeiro.

Campo	Tipo	Definição
percentual_metas_atingidas	DOUBLE	100 vezes a proporção de metas atingidas no grupo.
meta_media	DOUBLE	Média de targeted_productivity.
produtividade_media	DOUBLE	Média de actual_productivity.
minutos_extras_medios	DOUBLE	Média de over_time, em minutos.
inatividade_media	DOUBLE	Média de idle_time, cuja unidade não consta na fonte.
registros_wip_informado	BIGINT	Quantidade de registros não nulos de wip.
wip_medio_informado	DOUBLE	Média de wip somente nos registros informados; nula sem observações.

Nas tabelas Gold, registros, metas_atingidas e registros_wip_informado são contagens inteiras não negativas. As duas últimas não excedem registros. percentual_metas_atingidas varia de 0 a 100. As médias mantêm as unidades das medidas de origem; wip_medio_informado permanece nulo quando não há registros com wip informado.

4 Pipeline de Dados

O notebook executa a sequência CSV → Bronze → análise inicial de qualidade → Silver → Gold → consultas. A Bronze conserva os dados de entrada. Na Silver, date passa a DATE, team a INT e as medidas numéricas a DOUBLE. O texto do setor é aparado; a grafia sweing é registrada em setor_original e corrigida para sewing em setor. O cálculo atingiu_meta compara a produtividade realizada à meta linha a linha.

Antes de gravar a Silver, o código interrompe a execução se houver nulos nos campos obrigatórios, divergências entre data e dia da semana ou repetição da chave data, setor e equipe. Os 506 wip ausentes continuam nulos; tratá-los como zero alteraria o significado da medida. Os 37 valores de actual_productivity acima de 1 são conservados e sinalizados para exame.

```
numeric_columns = ["targeted_productivity", "smv", "wip", "over_time", "incentive", "idle_time", "idle_men", "no_of_style_change", "no_of_workers",
"actual_productivity"]
typed = br.select(*[F.col(c) for c in source_columns], "source_file")
typed = typed.withColumn("data_registro", F.to_date("date", "M/d/yyyy"))
typed = typed.withColumn("equipe", F.expr("try_cast(team as int)"))
typed = typed.withColumn("setor_original", F.col("department"))
typed = typed.withColumn("setor", F.when(F.lower(F.trim("department")) == "sweing", F.lit("sewing")).otherwise(F.lower(F.trim("department"))))
for field in numeric_columns:
    typed = typed.withColumn(field, F.expr(f"try_cast(trim(`{field}`) as double)"))
typed = typed.withColumn("atingiu_meta", F.col("actual_productivity") >= F.col("targeted_productivity"))
typed = typed.withColumn("produtividade_acima_de_um", F.col("actual_productivity") > 1)
typed = typed.withColumn("dia_confere", F.date_format("data_registro", "EEEE") == F.col("day"))

# Falhas de conversão dos campos obrigatórios interrompem o pipeline, sem perda silenciosa.
required = ["data_registro", "equipe", "targeted_productivity", "actual_productivity"]
bad_required = typed.filter(F.expr(" OR ".join(f"`{x}` IS NULL" for x in required))).count()
assert bad_required == 0, f"Campos obrigatórios inválidos em {bad_required} linhas"
bad_day = typed.filter(~F.col("dia_confere") | F.col("dia_confere").isNull()).count()
assert bad_day == 0, f"Dia da semana inconsistente em {bad_day} linhas"
duplicate_keys = typed.groupBy("data_registro", "setor", "equipe").count().filter("count > 1").count()
assert duplicate_keys == 0, f"Chaves repetidas após padronização: {duplicate_keys}"

silver = typed.drop("date", "department", "team")
silver.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(f"{TABLE_PREFIX}.silver_produtividade")
print("Linhas Silver:", spark.table(f"{TABLE_PREFIX}.silver_produtividade").count())
```

> [See performance \(5\)](#) ✓ Checked for improvements

> 2 dataframes: silver, typed

Linhas Silver: 1197

Figura 6 — Conversão, padronização e verificações da camada Silver, com 1.197 registros. Fonte: elaboração própria.

As três tabelas Gold calculam contagens, percentuais e médias por setor, por data e setor e por setor e equipe. A gravação em Delta com modo overwrite substitui as tabelas em cada execução e evita o acúmulo de linhas ao processar novamente o mesmo CSV. A tabela por setor soma 1.197 registros, quantidade igual à da Silver; as três Gold possuem, respectivamente, 2, 118 e 24 linhas.

```
def measures(grouped):
    return grouped.agg(
        F.count("*").alias("registros"),
        F.sum(F.col("atingiu_meta").cast("long")).alias("metas_atingidas"),
        F.round(100 * F.avg(F.col("atingiu_meta").cast("double")), 2).alias("percentual_metas_atingidas"),
        F.round(F.avg("targeted_productivity"), 4).alias("meta_media"),
        F.round(F.avg("actual_productivity"), 4).alias("produtividade_media"),
        F.round(F.avg("over_time"), 2).alias("minutos_extras_medios"),
        F.round(F.avg("idle_time"), 2).alias("inatividade_media"),
        F.count("wip").alias("registros_wip_informado"),
        F.round(F.avg("wip"), 2).alias("wip_medio_informado"),
    )

gold = {
    "gold_produtividade_setor": measures(s.groupBy("setor")),
    "gold_produtividade_dia_setor": measures(s.groupBy("data_registro", "setor")),
    "gold_produtividade_equipe": measures(s.groupBy("setor", "equipe")),
}
for name, frame in gold.items():
    frame.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable(f"{TABLE_PREFIX}.{name}")
    print(name, spark.table(f"{TABLE_PREFIX}.{name}").count(), "linhas")
assert spark.table(f"{TABLE_PREFIX}.gold_produtividade_setor").agg(F.sum("registros")).first()[0] == s.count()

> See performance \(8\)
```

gold_produtividade_setor 2 linhas
gold_produtividade_dia_setor 118 linhas
gold_produtividade_equipe 24 linhas

Figura 7 — Criação das tabelas Gold por setor, por dia e setor, e por equipe. Fonte: elaboração própria.

5 Qualidade de Dados

A conferência inicial examinou a completude de cada coluna, a consistência de categorias e datas, a duplicação de linhas e chaves e os limites observados nas medidas numéricas. Na execução em nuvem, a Bronze apresentou 1.197 linhas e nenhuma linha inteiramente duplicada. A Silver manteve as 1.197 linhas após conversão, com validações de campos obrigatórios, dia da semana e chave composta concluídas sem falhas.

```
br = spark.table(f"{TABLE_PREFIX}.bronze_produtividade")
for field in source_columns:
    missing = br.filter(F.col(field).isNull() | (F.trim(F.col(field)) == "")).count()
    distinct = br.select(field).distinct().count()
    print(f"{field}: ausentes={missing}, distintos={distinct}")
print("Linhas completamente duplicadas:", br.groupBy(*source_columns).count().filter("count > 1").agg(F.sum(F.col("count") - 1)).first()[0] or 0)
display(br.groupBy("department").agg(F.count("*").alias("linhas"), F.sum(F.when(F.col("wip").isNull() | (F.trim("wip") == ""), 1).otherwise(0)).alias("wip_ausente")))
```

> [See performance \(32\)](#) ✓ Checked for improvements

```
br: pyspark.sql.connect.dataframe.DataFrame
  date: string
  quarter: string
  department: string
  day: string
  team: string
  targeted_productivity: string
  smv: string
  wip: string
  over_time: string
  incentive: string
  idle_time: string
  idle_men: string
  no_of_style_change: string
  no_of_workers: string
  actual_productivity: string
  source_file: string
  ingested_at: timestamp
```

Figura 8 — Código do perfil inicial da Bronze. Fonte: elaboração própria.

```

date: ausentes=0, distintos=59
quarter: ausentes=0, distintos=5
department: ausentes=0, distintos=3
day: ausentes=0, distintos=6
team: ausentes=0, distintos=12
targeted_productivity: ausentes=0, distintos=9
smv: ausentes=0, distintos=70
wip: ausentes=506, distintos=549
over_time: ausentes=0, distintos=143
incentive: ausentes=0, distintos=48
idle_time: ausentes=0, distintos=12
idle_men: ausentes=0, distintos=10
no_of_style_change: ausentes=0, distintos=3
no_of_workers: ausentes=0, distintos=61
actual_productivity: ausentes=0, distintos=879
Linhas completamente duplicadas: 0

```

Table  

	A ^B C department	1 ² ₃ linhas	1 ² ₃ wip_ausente
1	sweing	691	0
2	finishing	257	257
3	finishing	249	249

Figura 9 — Resultado do perfil inicial da Bronze. Fonte: elaboração própria.

A única coluna original com ausências é wip: 506 registros, equivalentes a 42,27% do total. Todas as ausências pertencem a finishing; os 691 registros de sewing possuem wip informado. Isso restringe a comparação dessa medida: não há média observada de wip para finishing. Por essa razão, as tabelas Gold apresentam tanto a média quanto a contagem que a sustenta.

A fonte possui três grafias do departamento: sweing (691 linhas), finishing (249) e finishing com espaço final (257). Após retirar espaços e corrigir sweing, há dois setores. O arquivo também contém Quarter5, embora a descrição resumida da UCI diga que o mês se divide em quatro partes; a categoria original foi mantida, sem tratá-la como trimestre do ano.

A produtividade realizada vai de 0,233705476 a 1,1204375. Como a descrição da UCI menciona intervalo de 0 a 1, os 37 valores acima de 1 foram sinalizados, sem truncamento. O maior wip informado é 23.122 e o maior over_time é 25.920 minutos. São valores que merecem investigação com informações adicionais do processo; os dados disponíveis não permitem classificá-los como erros de medição.

Para examinar valores extremos nos dez atributos numéricos, foi aplicado o critério do intervalo interquartil (IQR): valores abaixo de $Q1 - 1,5 \times IQR$ ou acima de $Q3 + 1,5 \times IQR$ foram sinalizados para investigação. Quando $Q1$ e $Q3$ eram iguais a zero, o critério não foi aplicado, pois sinalizaria qualquer valor diferente de zero. Os registros sinalizados foram preservados; a identificação estatística de um extremo não comprova erro de medição.

Atributo	Limites IQR aproximados	Registros sinalizados
targeted_productivity	0,55 a 0,95	79
smv	-26,54 a 54,74	0
wip	51,50 a 1.975,50	21
over_time	-6.840,00 a 15.240,00	1
incentive	-75,00 a 125,00	11
idle_time	IQR = 0; critério não aplicado	—

Atributo	Limites IQR aproximados	Registros sinalizados
idle_men	IQR = 0; critério não aplicado	—
no_of_style_change	IQR = 0; critério não aplicado	—
no_of_workers	−63,00 a 129,00	0
actual_productivity	0,35 a 1,15	54

```
s = spark.table(f"{TABLE_PREFIX}.silver_produtividade")
assert s.count() == br.count(), "A transformação alterou o número de linhas"
print("WIP nulo:", s.filter(F.col("wip").isNull()).count())
print("Produtividade > 1:", s.filter("produtividade_acima_de_um").count())
for field in numeric_columns:
    values = s.agg(F.count(F.col(field)).alias("informados"), F.min(field).alias("minimo"), F.max(field).alias("maximo")).first()
    print(field, values.asDict())
print("Categorias de trimestre do mês e de setor:")
display(s.groupBy("quarter", "setor").count().orderBy("quarter", "setor"))
```

> [See performance \(15\)](#)

> s: pyspark.sql.connect.dataframe.DataFrame = [quarter: string, day: string ... 18 more fields]

WIP nulo: 506
Produtividade > 1: 37
targeted_productivity {'informados': 1197, 'minimo': 0.07, 'maximo': 0.8}
smv {'informados': 1197, 'minimo': 2.9, 'maximo': 54.56}
wip {'informados': 691, 'minimo': 7.0, 'maximo': 23122.0}
over_time {'informados': 1197, 'minimo': 0.0, 'maximo': 25920.0}
incentive {'informados': 1197, 'minimo': 0.0, 'maximo': 3600.0}
idle_time {'informados': 1197, 'minimo': 0.0, 'maximo': 300.0}
idle_men {'informados': 1197, 'minimo': 0.0, 'maximo': 45.0}
no_of_style_change {'informados': 1197, 'minimo': 0.0, 'maximo': 2.0}
no_of_workers {'informados': 1197, 'minimo': 2.0, 'maximo': 89.0}
actual_productivity {'informados': 1197, 'minimo': 0.233705476, 'maximo': 1.1204375}
Categorias de trimestre do mês e de setor:

Table

	Ⓐ quarter	Ⓐ setor	Ⓐ count
1	Quarter1	finishing	149
2	Quarter1	sewing	211
3	Quarter2	finishing	147
4	Quarter2	sewing	188
5	Quarter3	finishing	81
6	Quarter3	sewing	129
7	Quarter4	finishing	108
8	Quarter4	sewing	140
9	Quarter5	finishing	21
10	Quarter5	sewing	23

Figura 10 — Verificações da Silver: valores ausentes, produtividade acima de 1 e limites das medidas. Fonte: elaboração própria.

```

# Triagem de valores extremos pelo intervalo interquartil (IQR).
# Registros sinalizados são candidatos à investigação, não erros confirmados.
from pyspark.sql import functions as F

s = spark.table("workspace.mvp_produtividade.silver_produtividade")

numeric_columns = [
    "targeted_productivity", "smv", "wip", "over_time",
    "incentive", "idle_time", "idle_men", "no_of_style_change",
    "no_of_workers", "actual_productivity"
]

for campo in numeric_columns:
    valores = s.where(F.col(campo).isNotNull())
    q1, q3 = valores.approxQuantile(campo, [0.25, 0.75], 0)
    iqr = q3 - q1
    if iqr == 0:
        print(f"{campo}: Q1={q1:.2f}; Q3={q3:.2f}; IQR=0; "
              "critério de extremos não aplicado")
        continue
    inferior = q1 - 1.5 * iqr
    superior = q3 + 1.5 * iqr
    extremos = valores.where(
        (F.col(campo) < inferior) | (F.col(campo) > superior)
    ).count()
    print(
        f"{campo}: Q1={q1:.2f}; Q3={q3:.2f}; "
        f"limites=[{inferior:.2f}, {superior:.2f}]; "
        f"candidatos={extremos}"
    )

```

> [See performance \(17\)](#)

> `s: pyspark.sql.connect.dataframe.DataFrame = [quarter: string, day: string ... 18 more fields]`

```

targeted_productivity: Q1=0.70; Q3=0.80; limites=[0.55, 0.95]; candidatos=79
smv: Q1=3.94; Q3=24.26; limites=[-26.54, 54.74]; candidatos=0
wip: Q1=773.00; Q3=1254.00; limites=[51.50, 1975.50]; candidatos=21
over_time: Q1=1440.00; Q3=6960.00; limites=[-6840.00, 15240.00]; candidatos=1
incentive: Q1=0.00; Q3=50.00; limites=[-75.00, 125.00]; candidatos=11
idle_time: Q1=0.00; Q3=0.00; IQR=0; critério de extremos não aplicado
idle_men: Q1=0.00; Q3=0.00; IQR=0; critério de extremos não aplicado
no_of_style_change: Q1=0.00; Q3=0.00; IQR=0; critério de extremos não aplicado
no_of_workers: Q1=9.00; Q3=57.00; limites=[-63.00, 129.00]; candidatos=0
actual_productivity: Q1=0.65; Q3=0.85; limites=[0.35, 1.15]; candidatos=54

```

Figura 11 — Triagem de valores extremos pelo método IQR: código e resultados para os atributos numéricos da Silver.
Fonte: elaboração própria.

6 Análise de Dados

Cumprimento das metas por setor

A meta foi atingida ou superada em 302 dos 506 registros de finishing, ou 59,68%, e em 573 dos 691 registros de sewing, ou 82,92%. A diferença é de 23,24 pontos percentuais. Embora a produtividade média realizada seja 0,753 em finishing e 0,722 em sewing, esse valor isolado não mede a frequência de cumprimento da meta, porque as metas variam entre os registros.

```
print("Pergunta 1: cumprimento das metas por setor")
display(spark.sql(f"SELECT * FROM {TABLE_PREFIX}.gold_produtividade_setor ORDER BY setor"))
```

Figura 12 — Consulta do cumprimento das metas por setor. Fonte: elaboração própria.

	setor	registros	metas atingidas	percentual_metas_atingidas	meta_media	produtividade_media
1	finishing	506	302	59.68	0.7374	0.753
2	sewing	691	573	82.92	0.724	0.722

Figura 13 — Registros, metas atingidas e produtividade por setor. Fonte: elaboração própria.

Variação ao longo do período e entre equipes

A Gold diária contém 118 grupos, correspondentes a 59 datas em cada setor. Para resumir a evolução sem calcular uma média simples de percentuais diários, a tabela abaixo divide o número de metas atingidas pelo total de registros de cada mês e setor. Os valores foram conferidos no CSV com a mesma regra de atingiu_meta utilizada pela Silver.

Mês de 2015	finishing	sewing
Janeiro	154/235 = 65,53%	269/307 = 87,62%
Fevereiro	92/174 = 52,87%	201/269 = 74,72%
Março, até o dia 11	56/97 = 57,73%	103/115 = 89,57%

```
print("Pergunta 2: evolução diária por setor e resultados por equipe")
display(spark.sql(f"SELECT * FROM {TABLE_PREFIX}.gold_produtividade_dia_setor ORDER BY data_registro, setor"))
display(spark.sql(f"SELECT * FROM {TABLE_PREFIX}.gold_produtividade_equipe ORDER BY setor, equipe"))
```

Figura 14 — Consultas dos resultados por dia e setor e por equipe. Fonte: elaboração própria.

Pergunta 2: evolução diária por setor e resultados por equipe

	data_registro	setor	registros	metas atingidas	percentual_metas_atingidas	meta_media	produtividade_media	minutos
1	2015-01-01	finishing	7	3	42.86	0.7357	0.6563	
2	2015-01-01	sewing	12	9	75	0.7625	0.7551	
3	2015-01-03	finishing	9	6	66.67	0.7778	0.8354	
4	2015-01-03	sewing	12	11	91.67	0.775	0.7783	
5	2015-01-04	finishing	9	5	55.56	0.7778	0.7719	
6	2015-01-04	sewing	12	10	83.33	0.775	0.7864	
7	2015-01-05	finishing	10	8	80	0.77	0.8159	
8	2015-01-05	sewing	12	12	100	0.775	0.7795	
9	2015-01-06	finishing	9	7	77.78	0.7889	0.8291	
10	2015-01-06	sewing	12	11	91.67	0.7792	0.7878	
11	2015-01-07	finishing	10	7	70	0.78	0.7591	
12	2015-01-07	sewing	12	11	91.67	0.7875	0.7722	
13	2015-01-08	finishing	10	6	60	0.78	0.7841	
14	2015-01-08	sewing	12	10	83.33	0.75	0.7405	
15								

Figura 15 — Exemplo dos resultados diários por setor. Fonte: elaboração própria.

	1.1 setor	1.2 equipe	1.3 registros	1.4 metas_atin...	1.5 percentual_metas_atingidas	1.6 meta_media	1.7 produtividade_media	1.8 minutos_ext
1	finishing	1	49	37	75.51	0.75	0.8278	
2	finishing	2	52	35	67.31	0.7529	0.7801	
3	finishing	3	37	31	83.78	0.7405	0.8473	
4	finishing	4	46	33	71.74	0.7185	0.8206	
5	finishing	5	35	28	80	0.66	0.7916	
6	finishing	6	35	11	31.43	0.7229	0.6279	
7	finishing	7	41	17	41.46	0.7305	0.6858	
8	finishing	8	53	19	35.85	0.7132	0.6927	
9	finishing	9	46	26	56.52	0.7761	0.7245	
10	finishing	10	43	26	60.47	0.7535	0.7234	
11	finishing	11	29	14	48.28	0.7362	0.6938	
12	finishing	12	40	25	62.5	0.7788	0.7923	
13	sewing	1	56	53	94.64	0.7438	0.8151	
14	sewing	2	57	47	82.46	0.7281	0.7624	

Figura 16 — Exemplo dos resultados por equipe. Fonte: elaboração própria.

Nos dois setores, a proporção de metas atingidas foi menor em fevereiro que em janeiro. Março inclui somente parte do mês e deve ser interpretado nessa condição. Nas agregações por equipe também há diferenças: a equipe 6 de finishing cumpriu 11 metas em 35 registros (31,43%), enquanto a equipe 1 de sewing cumpriu 53 em 56 (94,64%). Essas diferenças descrevem os grupos observados; não isolam tipo de tarefa, composição das equipes ou dificuldade das metas.

Indicadores operacionais segundo o cumprimento da meta

Em sewing, os 118 registros que não atingiram a meta tiveram média de idle_time de 5,31, ante 0,43 nos 573 registros que a atingiram. As médias de over_time foram, respectivamente, 6.394,58 e 6.531,61 minutos. Entre os registros com wip informado, as médias foram 845,58 e 1.261,49 itens, respectivamente.

Em finishing, 204 registros não atingiram a meta e 302 atingiram. O over_time médio foi, respectivamente, 1.849,41 e 1.962,91 minutos; o idle_time médio foi zero em ambos os grupos. Nenhum registro desse setor informa wip. Essas comparações mostram associações descritivas dentro do período analisado; não estabelecem que o tempo extra, a inatividade ou o trabalho em andamento tenham causado o resultado de produtividade.

```
print("Pergunta 3: indicadores operacionais por setor e cumprimento de meta")
display(s.groupBy("setor", "atingiu_meta").agg(
  F.count("*").alias("registros"), F.round(F.avg("over_time"), 2).alias("minutos_extras_medios"),
  F.round(F.avg("idle_time"), 2).alias("inatividade_media"),
  F.count("wip").alias("registros_wip_informado"), F.round(F.avg("wip"), 2).alias("wip_medio_informado")
).orderBy("setor", "atingiu_meta"))
```

Figura 17 — Código da consulta dos indicadores operacionais por setor e cumprimento da meta. Fonte: elaboração própria.

Pergunta 3: indicadores operacionais por setor e cumprimento de meta

	1.1 setor	1.2 atingiu_meta	1.3 registros	1.4 minutos_extras_medios	1.5 inatividade_media	1.6 registros_wip_informado	1.7 wip_medio_informado
1	finishing	false	204	1849.41	0	0	null
2	finishing	true	302	1962.91	0	0	null
3	sewing	false	118	6394.58	5.31	118	845.58
4	sewing	true	573	6531.61	0.43	573	1261.49

Figura 18 — Indicadores operacionais por setor e cumprimento da meta. Fonte: elaboração própria.

Síntese das perguntas e respostas

1. Qual proporção de registros atinge ou supera a meta de produtividade em cada setor?

Em finishing, a meta foi atingida em 302 de 506 registros (59,68%). Em sewing, foram 573 de 691 (82,92%). A diferença é de 23,24 pontos percentuais. Trata-se da frequência de cumprimento da meta em cada setor, não de uma comparação direta das médias de produtividade, pois as metas variam entre os registros.

2. Como a produtividade realizada e o cumprimento da meta variam entre equipes e ao longo das datas disponíveis?

As tabelas Gold permitem examinar a produtividade média e o cumprimento da meta por data e setor e por setor e equipe. Nos dois setores, a proporção de metas atingidas foi menor em fevereiro do que em janeiro: passou de 65,53% para 52,87% em finishing e de 87,62% para 74,72% em sewing. Entre as equipes mostradas nos resultados, a equipe 6 de finishing atingiu 11 metas em 35 registros (31,43%), enquanto a equipe 1 de sewing atingiu 53 em 56 (94,64%). Nesse recorte, as produtividades médias realizadas dessas equipes foram 0,6279 e 0,8151, respectivamente; em 1º de janeiro, as médias diárias foram 0,6563 em finishing e 0,7551 em sewing. Março contém somente parte do mês, e esses resultados não isolam diferenças nas tarefas ou nas metas atribuídas.

3. Como se distribuem horas extras, tempo de inatividade e trabalho em andamento nos registros que atingem e não atingem a meta?

Em sewing, o idle_time médio foi 5,31 nos registros que não atingiram a meta e 0,43 nos que a atingiram. O over_time médio foi, respectivamente, 6.394,58 e 6.531,61 minutos; o wip médio entre registros informados foi 845,58 e 1.261,49 itens. Em finishing, o idle_time médio foi zero nos dois grupos e nenhum registro informou wip. São comparações descritivas: elas não demonstram que esses indicadores causaram o resultado da meta.

Considerações finais

As consultas mostram como o pipeline permite passar de registros individuais para diferentes perspectivas de análise: setor, período, equipe e indicadores operacionais. Os resultados ajudam a identificar grupos que merecem exame mais detalhado, mas precisam ser interpretados à luz das limitações da base. Sem informações adicionais sobre tarefas, condições de operação e definição das metas, as diferenças observadas não permitem atribuir causas nem avaliar isoladamente o desempenho das equipes.

A preservação dos valores de origem foi importante para essa interpretação. As ausências de wip permaneceram nulas, e as produtividades acima de 1 e os valores extremos foram sinalizados sem descarte automático. Assim, as decisões de tratamento ficam explícitas e podem ser revistas caso novas informações sobre a coleta se tornem disponíveis.

Pontos de melhoria

Uma próxima etapa seria reunir informações sobre os processos de trabalho e os critérios das metas para investigar os grupos destacados nas consultas. Também seria útil esclarecer a ausência de wip em finishing e verificar os valores extremos com quem conhece a operação que gerou os registros. Na parte técnica, a carga manual e estática do CSV poderia evoluir para atualizações periódicas acompanhadas de verificações de qualidade.

7 Autoavaliação

O objetivo definido para este MVP foi atingido: os dados públicos foram carregados no Databricks, preservados na Bronze, tratados na Silver e organizados em tabelas Gold que sustentam as consultas apresentadas. A execução e as evidências documentadas permitem acompanhar o caminho dos dados desde o arquivo recebido até os resultados analisados.

As principais dificuldades estiveram na carga do arquivo para a plataforma, na conferência das tabelas persistidas e nas decisões de tratamento dos dados. Foi necessário distinguir ausência de valor de valor zero, padronizar a identificação do setor sem perder a informação original e registrar ocorrências que exigem investigação sem classificá-las automaticamente como erros. Essas escolhas tornaram a análise mais verificável.

O trabalho entrega uma solução funcional para as perguntas propostas, dentro do alcance do conjunto de dados utilizado. Seu limite principal é a falta de contexto operacional para explicar as diferenças encontradas. As melhorias indicadas nas considerações finais oferecem um caminho para ampliar o projeto sem apresentar as conclusões descritivas atuais como relações de causa e efeito.

Referências

UCI Machine Learning Repository. Productivity Prediction of Garment Employees. DOI: 10.24432/C51S6D. <https://archive.ics.uci.edu/dataset/597/productivity+prediction+of+garment+employees>

Dornelas, Izis. MVP de Engenharia de Dados: produtividade no chão de fábrica. Código e notebook exportado. <https://github.com/izisdornelas/mvp-engenharia-dados-produtividade>